

Buy or Wait?

Architecture and Design of a Verified-or-Fail-Closed Financial Decision Agent

HackerRank Orchestrate — September 2026

Revision: main 1f381fe (verifier sign-off on 7b68395)

Abstract

Buy or Wait? decides, for every purchase or payment request, how much is safe to pay today, whether the user can afford it now, with a plan, later, or not at all, and which payment method, schedule and spending changes to recommend. The system is a single Rust crate. A deterministic decision engine reconstructs each user's cash position from structured financial events, fixed dated exchange rates, supplied payment options and evidence from messages and receipt images, then forecasts the balance day by day against the user's minimum balance. Receipt images are read at ingestion by a self-hosted OCR model (`baidu/Unlimited-OCR` on vLLM) and cached. Every figure the engine uses from an image is mapped deterministically from printed labels and must pass a witness gate, or it fails closed and is never guessed. The final run produced 250 decisions, 0 contract violations, 0 wrong image figures and byte-identical output across two cold runs.

Contents

1	Goals and design principles	2
2	System context	2
3	Crate structure	2
4	Batch pipeline	3
5	Image evidence pipeline	3
5.1	Why OCR plus deterministic mapping	3
5.2	OCR serving configuration	4
5.3	Label to role mapping	4
5.4	Acceptance rules (user rulings)	5
5.5	Results on the 16 images	5
6	Decision engine	6
6.1	Ledger and cash rules	6
6.2	Forecast	6
6.3	Plans and ranking	7
6.4	Explanations	7
7	Message evidence	7
8	Verification and ship gate	8
9	Requirements traceability	8
10	Operations	9
11	Known limitations and next steps	9

1 Goals and design principles

1. **Contract-exact output.** One row per `request_id` in `dataset/requests.csv`, with the exact 8-column header, enumerations and field semantics of `AGENTS.md §6.2`.
2. **Verified or fail closed.** No figure enters the ledger unless it is proven. An unproven image or message figure is dropped with a recorded reason, never estimated. Wrong figures must be 0.
3. **Determinism.** Two -cold runs produce byte-identical `output.csv`. Money is fixed-point, models run at temperature 0, and every model output is cached with a config hash.
4. **Models read, Rust decides.** Models only transcribe. Every financial interpretation (which figure, cash treatment, plan ranking, explanation) is deterministic Rust.
5. **Conservative safety.** The balance never falls below `minimum_balance_to_keep` after any projected essential expense or recommended payment. Pending debits are reserved; unsettled credits are not counted.
6. **Operable and auditable.** Terminal-runnable, secrets from environment only, a per-request `DecisionFacts` record, persisted provenance for every image read, and a usage report for the final run.

2 System context

The product shape is a two-phase system. Users upload receipts and documents first; ingestion OCRs them once and caches the result. Questions (“*can I afford this?*”) are then answered from the cached, verified evidence. The submission runs both phases in batch over the dataset.

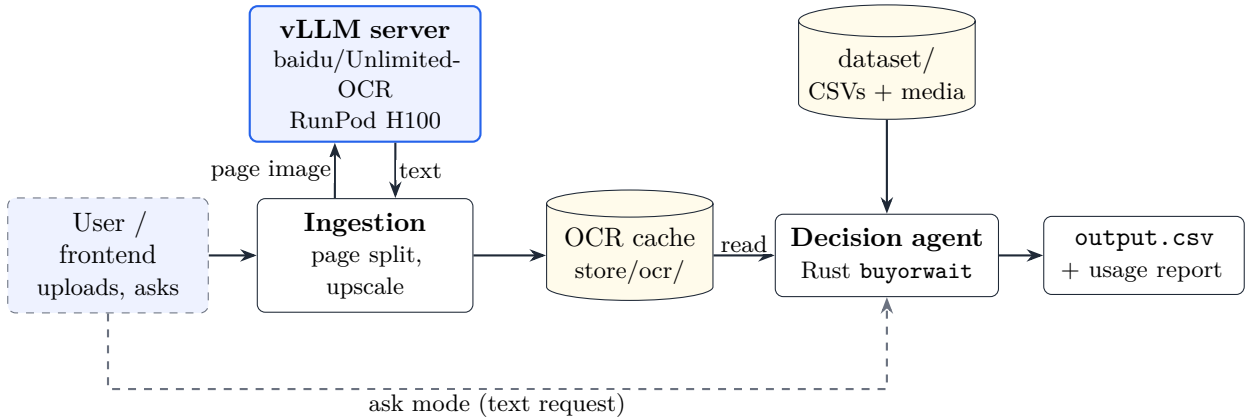


Figure 1: Context. The OCR model sits behind an OpenAI-compatible vLLM endpoint and is called only at ingestion. The decision agent reads the cache.

3 Crate structure

Module	Responsibility
<code>engine::money</code>	Fixed-point money in 10^{-4} currency units; rounding only at output.
<code>engine::rules</code>	All tunable decision rules and toggles (<code>Rules::default()</code>).
<code>engine::session</code>	One user: profile, events, rates; entry point <code>Session::from_model</code> .
<code>engine::ledger</code>	Cash rules by status and direction, lifecycle de-duplication, evidence amendments in the spec’s conflict order, FX conversion, <code>Fact</code> application.

Module	Responsibility
<code>engine::recurrence</code>	Detects recurring streams (salary, bills, variable spend) only where history supports them.
<code>engine::forecast</code>	Day-by-day balance projection, safe amount and earliest full-payment date.
<code>engine::plans</code>	Candidate plans (full, installments, two-payment partial, wait, spending changes), eligibility, safety replay, ranking.
<code>engine::facts</code>	DecisionFacts : every number the decision rests on.
<code>engine::explain</code>	Template explanations rendered only from DecisionFacts (0 tokens).
<code>extract::messages</code>	Typed message records and a deterministic parser for known message families.
<code>extract::grounding</code>	A number claimed from a message must literally appear in its text.
<code>extract::retrieval</code>	Per-user, date-bounded message evidence retrieval.
<code>extract::ocr</code>	vLLM client, page splitting, conditional upscaling, config-hashed cache, usage.
<code>extract::ocr_parse</code>	Raw OCR (HTML tables, <code>< det ></code> blocks) into label:value rows, including zipping multi-value cells.
<code>extract::labels</code>	Deterministic keyword to role mapping into ImageFigures .
<code>extract::normalize</code>	Trap-aware amount, currency and date parsing (lakh grouping, comma decimals, Rs/Ps splits).
<code>extract::witness</code>	Witness identities, amount-in-words parser, contradiction checks.
<code>extract::images</code>	Selector by event type and status, witness gate, provenance (ImageReadProvenance).
<code>extract::intake</code>	ask-mode free-text request intake (not used in batch).
<code>store::cache / processed</code>	Model-call cache and persisted preprocessing (ledgers, image reads).
<code>evaluation::*</code>	Contract validator, invariants, sample scorer, replay mirror, false-accept gate, image agreement, hardcode scan, sign-off.

4 Batch pipeline

5 Image evidence pipeline

Sixteen financial events have blank amounts whose values are printed on images (receipts, bills, a payslip, a handwritten pharmacy bill). Four of them are pending or scheduled and therefore change a forecast.

5.1 Why OCR plus deterministic mapping

An audit of earlier vision-language-model reads (Qwen3-VL-235B, gemma-4-31B; that path has since been removed from the codebase) found that the models transcribed every printed final amount correctly. Every failure came from asking the model to *interpret* the page into fixed JSON keys: it invented totals, subtotals, balances and due-date cutoffs the page did not print. The redesign lets the model only transcribe what is printed; Rust assigns meaning from the printed label text.

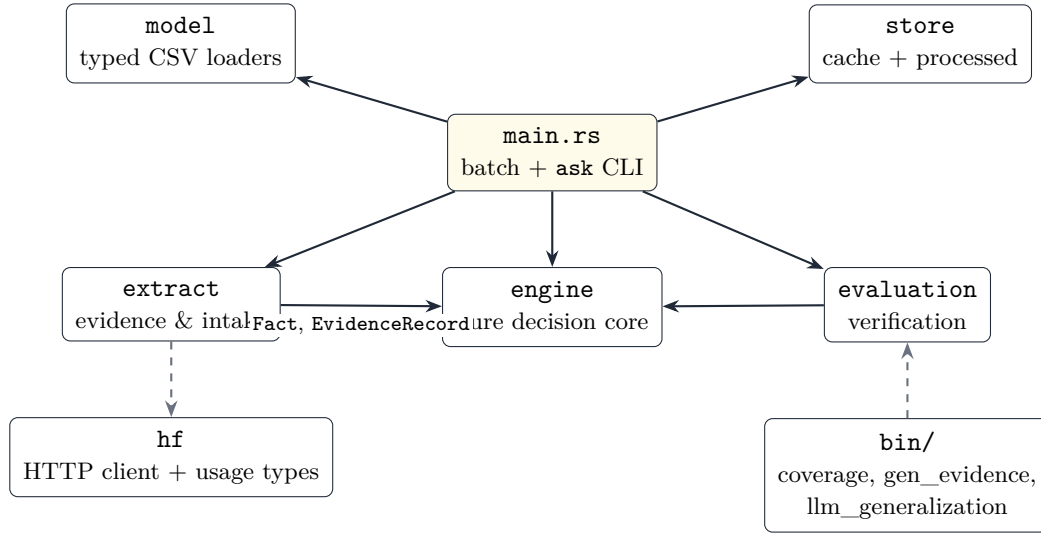


Figure 2: Module dependencies. **engine** has no I/O, network or model calls. **extract** converts evidence into the engine’s own **Fact** contract and never decides cash treatment.

5.2 OCR serving configuration

Model	baidu/Unlimited-OCR (MIT, 3.34B parameters, BF16; DeepSeek-OCR lineage)
Server	vllm/vllm-openai:unlimited-ocr-cu129, <code>-trust-remote-code</code> , <code>-logits_processors ...NGramPerReqLogitsProcessor</code> , prefix and multimodal caches disabled
Request	one image per request; <code>temperature=0</code> , <code>max_tokens=8192</code> , <code>skip_special_tokens=false</code> , <code>vllm_xargs={ngram_size:35, window_size:128}</code> ; explicit User-Agent (the RunPod proxy rejects requests without one)
Environment	OCR_BASE_URL, OCR_MODEL, optional OCR_API_KEY
Cache key	config hash over model, prompt, token limit, n-gram arguments, page scale and split thresholds
Final cold run	17 pages (16 images; one stitched two-page invoice), 31,309 prompt + 11,703 completion = 43,012 tokens, 53.6s wall time; two runs byte-identical over all 16 images

5.3 Label to role mapping

`extract::labels` maps each printed label with a fixed keyword list. No model decides meaning.

Role	Printed-label keywords (examples)
<code>grand_total</code>	Grand Total
<code>total</code>	Total, Total Amount, Total Bill Amount, Net Amount, Total paid, Total Amount Received
<code>amount_paid</code>	Amount Received, Amount Paid, Total paid
<code>balance_due /</code> <code>amount_due</code>	Balance Due, Balance / Amount Payable, Amount Due
<code>subtotal</code>	Sub Total, Item Bill, Item Total, Taxable Value (never promoted to total)
<code>tax</code>	CGST, SGST, IGST, GST, Tax, Taxes, Cess, VAT (lines summed in Rust)
<code>payslip</code>	Total Earnings, Total Deductions, Net Pay
<code>cutoff</code>	only <i>Amount due till/by/before</i> <date> and <i>Amount due after</i> <date>
<code>amount_in_words</code>	a words value (excluding <i>Paid amount in words</i>)
<code>ignored</code>	Cash Paid / tendered, Change, Payments (prior balance), bare Due Date

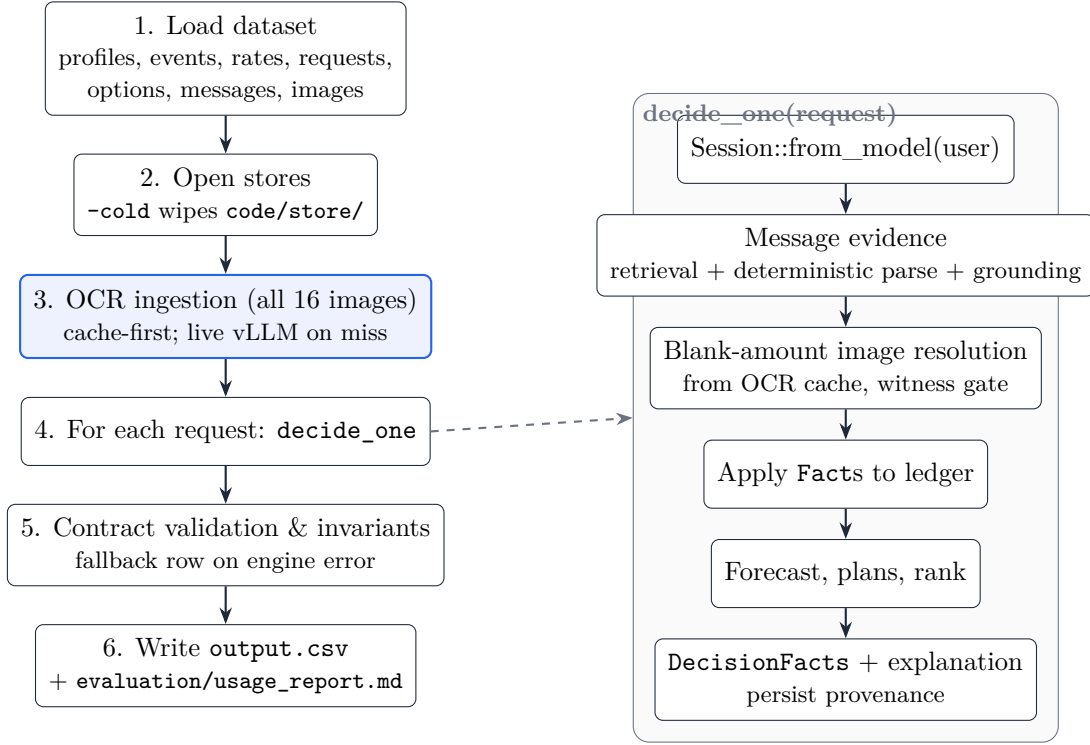


Figure 3: Batch control flow (`code/src/main.rs`). Requests are independent; per-user state is rebuilt for each request.

5.4 Acceptance rules (user rulings)

Witness identities (`extract::witness::WitnessKind`): line-item sum, subtotal plus charges, subtotal plus tax, gross minus deductions, paid plus balance, total minus paid, amount in words, repeated final label, after-cutoff amount exceeding a witnessed before-cutoff amount, line-item sum witnessed by words or label, and printed-final-label-only.

5.5 Results on the 16 images

Image	Event	Status	Accepted figure	Witness / reason	Scope
01	net salary (IDR)	settled	4,365,000	gross – deductions	sample
02	outstanding rent balance	scheduled	100,000	total – paid	sample
03	bulk groceries	settled	41,272	repeated final label	sample
04	delivered grocery order	settled	—	fail_closed: no final label (cropped)	sample
05	outstanding telecom bill	pending	822.05	after-cutoff > witnessed before-cutoff	sample
06	grocery tax invoice	settled	1,995	item sum witnessed by words	eval
07	restaurant tax invoice	settled	8,528	grand total	eval
08	property maintenance	settled	15,339	line-item sum	eval
09	water bill	settled	723	line-item sum	eval
10	large grocery invoice (2 pages)	pending	79,679.26	amount in words	eval
11	hospital bill	scheduled	3,650	repeated final label	eval
12	taxi fare	settled	33.50 USD	subtotal + tax	eval
13	tote bag order	settled	2,298	repeated final label	eval
14	pharmacy (handwritten)	settled	4,543	printed final label only	eval
15	airline ticket	settled	9,968	line-item sum	eval
16	EV charging	settled	393.22	amount in words	eval

All accepted figures match the rulings recorded before the final run; 0 false accepts.

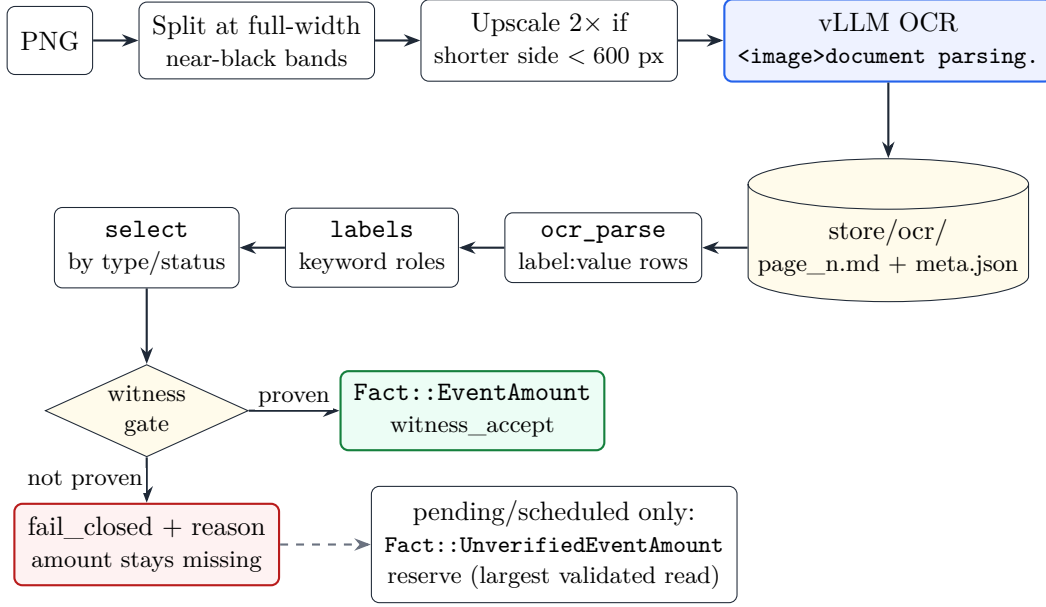


Figure 4: Image pipeline. OCR runs once per page at ingestion; everything after the cache is deterministic.

6 Decision engine

6.1 Ledger and cash rules

- **Settled** rows are already inside `current_balance`. **Pending** debits are reserved; pending credits, bonuses, commissions, refunds, lottery proceeds and unrealized investment value are never counted until they settle.
- **Scheduled** rows and confirmed salary count on their settlement dates. Linked lifecycle chains are de-duplicated, because a link alone never decides cash treatment.
- **Foreign currency** converts with the `exchange_rates.csv` row for the settlement date and stated direction. Converted amounts keep full precision; rounding happens only at output.
- **Evidence conflicts** resolve in the specified order: an explicit cancellation, settlement or amendment, then newer records from the same source, then a settled event, then the financially safer interpretation.
- **Percentage changes** (e.g. “rent +12%”) scale only recurring cycle rows, never a one-off scheduled balance.

6.2 Forecast

Let b_t be the projected end-of-day balance on day t of the horizon, ℓ_t the intraday low (debits applied before that day’s credits), M the minimum balance to keep and R the requested amount. With headroom $H_t = \ell_t - M$:

$$\text{safe} = \min(R, \max(0, \min_{t \geq 0} H_t)), \quad E = \min\left\{d : \min(b_d, \min_{t > d} \ell_t) - M \geq R\right\}.$$

A payment on day d lowers every later balance by the same amount, so suffix minima are computed once, making E linear-time. Recurring variable spending is projected conservatively. Three rule toggles, selected by held-out A/B, are on by default: fixed bills after salary credit on the same day, scheduled-row replacement by lifecycle or amount, and a mid-step phase for long-interval variable streams.

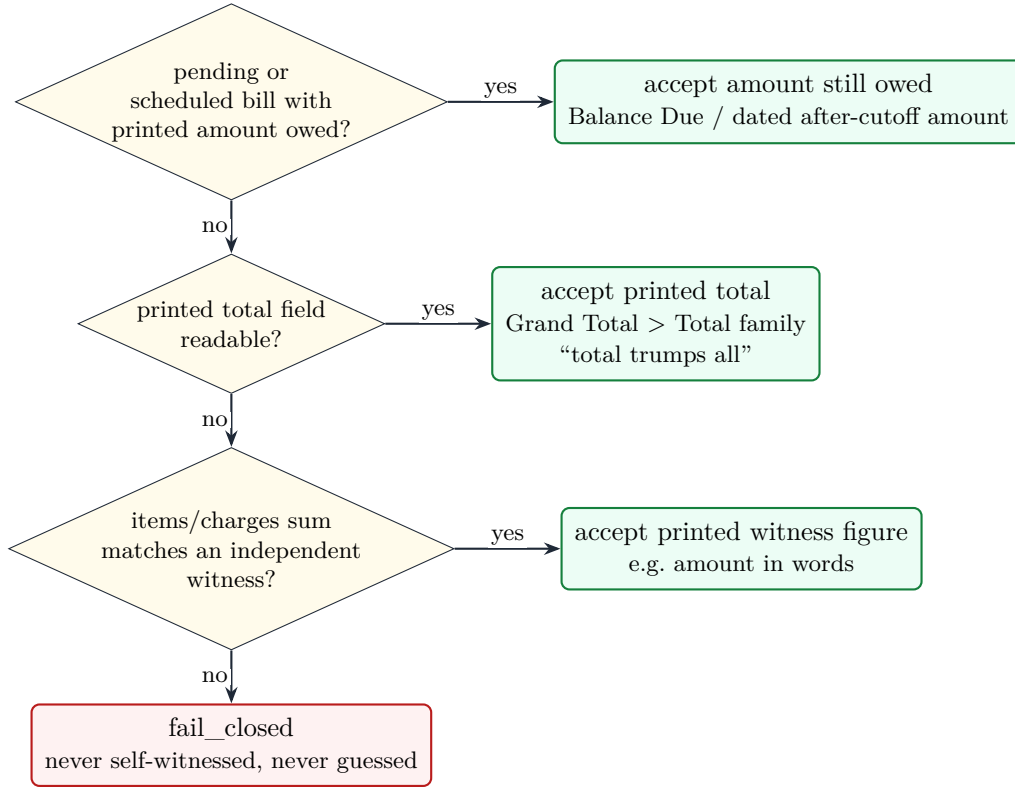


Figure 5: Figure selection. A detailed breakdown that does not sum never rejects a printed final amount (pages are often cropped). Cross-read and cutoff agreement checks remain in the gate.

6.3 Plans and ranking

Candidates are a full payment today, each supplied installment option, a two-payment partial plan (`amount_safe_to_pay` today, the remainder on E , only if partial payment is allowed and $E \leq$ desired completion date), wait until E , and variants with up to three spending changes on non-protected, flexible events in permitted categories. Every candidate is replayed day by day against the forecast. Candidates that would breach M , violate user preferences or `max_installment_months`, or miss the deadline are dropped with a recorded reason. Survivors rank lexicographically: completes by the deadline, avoids spending changes, minimizes total payment cost, starts earlier, uses fewer payments.

6.4 Explanations

`DecisionFacts` records every number the recommendation rests on: balance, reserved pending debits, the forecast trough and its drivers, accepted and unverified evidence, and plan rejections. `engine::explain` renders the explanation from templates filled only from these facts, so the explanation cannot contradict the row and costs 0 tokens.

7 Message evidence

Messages are untrusted evidence. `extract::retrieval` selects the user’s messages dated on or before the request date. `extract::messages` converts known message families into typed records with a deterministic parser; the final run made no language-model calls for messages. Every number a record claims must literally appear in the message text (`extract::grounding`), and embedded instructions are never executed. Records become engine **Facts**: amendment, cancellation, settlement, confirmed income, expense amount change. Facts with no matching **Fact** variant are dropped, never guessed.

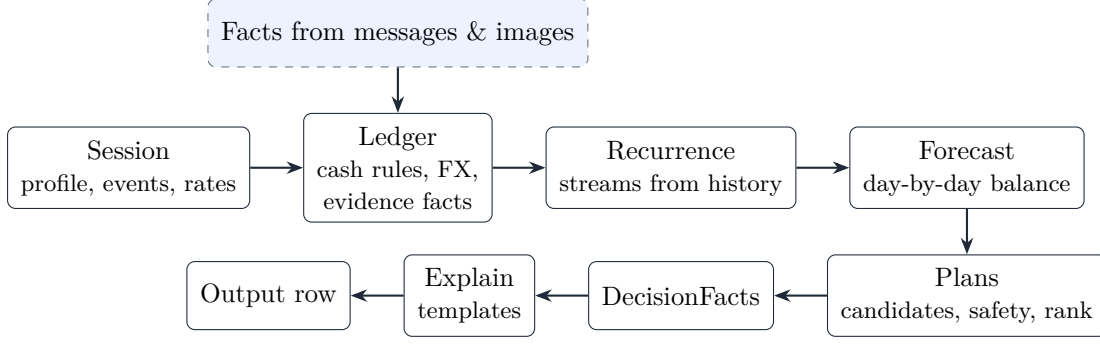


Figure 6: Engine flow (`code/src/engine`). Pure functions of the inputs; no I/O.

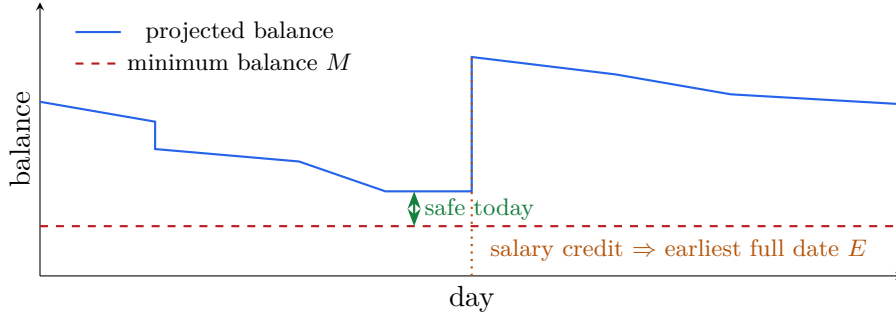


Figure 7: Illustration. The safe amount is the minimum headroom over the horizon. E is the first day from which the headroom never drops below R .

8 Verification and ship gate

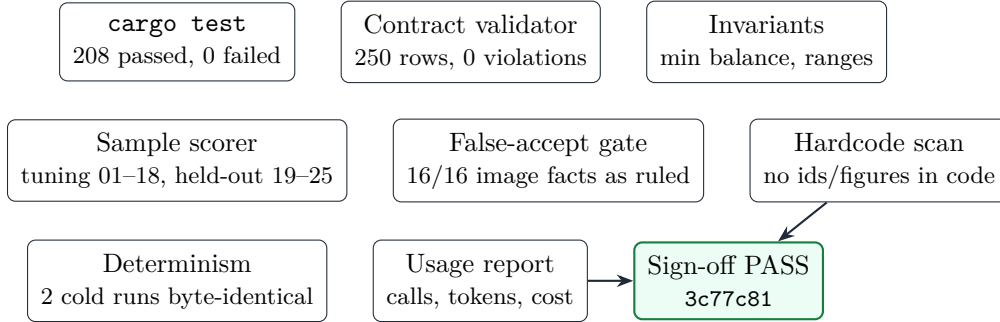


Figure 8: The ship gate (`evaluation::signoff`) requires every check to pass on one commit.

The sample scorer keeps held-out labels out of the tuning loop: held-out rows report pass counts only. Every rule change shipped as a separate commit with a moved-row check against the previous output.

9 Requirements traceability

Requirement (<code>AGENTS.md</code> / problem)	Implementation		Evidence
One row per request, exact header	<code>main.rs</code>	writer	+ 250/250, 0 violations
<code>amount_safe_to_pay</code> in $[0, R]$	<code>evaluation::contract</code>	forecast clamp	validator

Requirement (AGENTS.md / problem)	Implementation	Evidence
affordable_now $\Rightarrow E =$ request date	forecast E	validator
Partial plan: exactly two payments summing to R	plans candidate rules	validator
Installments follow a supplied option	plans request_payment_options.csv	over validator
≤ 3 spending changes, permitted events only	plans + profile categories	validator
Never below minimum balance	per-day replay	invariants
Reserve pending debits; ignore unsettled credits	ledger cash rules	scenario tests
Confirmed salary on settlement date	ledger, recurrence	scenario tests
FX by settlement-date row	ledger	scenario tests
Conflict resolution order	ledger::apply_evidence	scenario tests
Messages and images untrusted	grounding + witness gate	false-accept gate
Deterministic	fixed-point money, temp 0, config- hashed cache	byte-identical runs
Secrets from environment only	.env loader; zip excludes .env	zip scan
No organizer files or hardcoded labels	evaluation::hardcode_scan	clean
Usage report	evaluation/usage_report.md	present

10 Operations

```
# 1) OCR server (any NVIDIA host; POC used a RunPod H100)
docker run --rm --gpus all --network host --ipc host \
  vllm/vllm-openai:unlimited-ocr-cu129 baidu/Unlimited-OCR --trust-remote-code \
  --logits_processors vllm.model_executor.models.unlimited_ocr:NGramPerReqLogitsProcessor \
  --no-enable-prefix-caching --mm-processor-cache-gb 0 --host 0.0.0.0 --port 8000

# 2) Environment (code/.env, never committed)
OCR_BASE_URL=http://<host>:8000/v1

# 3) Run (from code/)
CARGO_TARGET_DIR=target cargo run --release -- --cold # submission run
CARGO_TARGET_DIR=target cargo run --release -- --requests ../dataset/sample_requests.csv
--out /tmp/s.csv
CARGO_TARGET_DIR=target cargo test
```

Ingestion is cache-first: pages already in `code/store/ocr/` under the same config hash are reused, and only missing pages call the endpoint. `--cold` wipes the processed store so every OCR call in the usage report is real. `HF_TOKEN` is not required for the default configuration.

11 Known limitations and next steps

- **Cropped documents.** Image 04 has no printed total. It fails closed by design and its settled amount stays unknown.
- **Serving-stack variance.** vLLM and the transformers reference implementation differ on a few

table cells (image 06's total cell, image 14's split total). Both are deterministic, and the witness rules recover the figures. A resolver model that only points at printed rows (Rust re-verifies) is a planned extension.

- **Handwriting.** Line items on image 14 are misread; only its printed total is used.
- **Unauthenticated POC endpoint.** Production needs an API key (`OCR_API_KEY`) and private networking.
- **Small labelled sample.** 25 sample requests (7 held out) limit how precisely rule toggles can be validated.